

SHARIF
UNIVERSITY OF TECHNOLOGY



Signals & Systems

Computer Assignment 1

Aria Fateh Kerdari
403110382

May 23, 2026

1 Introduction to MATLAB: Signal Representation

1.1 Plotting Discrete and Continuous Signals

In this section, various discrete and continuous signals were defined and plotted. For discrete signals, the `stem` function was utilized, while continuous signals were plotted using the `plot` function.

To approximate continuous signals in MATLAB, a sufficiently small sampling step (e.g., $dt = 0.01$) was chosen to ensure smooth transitions without visual aliasing. For the continuous convolution $x_7(t) = x_5(t) * x_6(t)$, the step size must be carefully managed to approximate the integral $y(t) = \int x_5(\tau)x_6(t-\tau)d\tau$, meaning the discrete convolution result was scaled by dt .

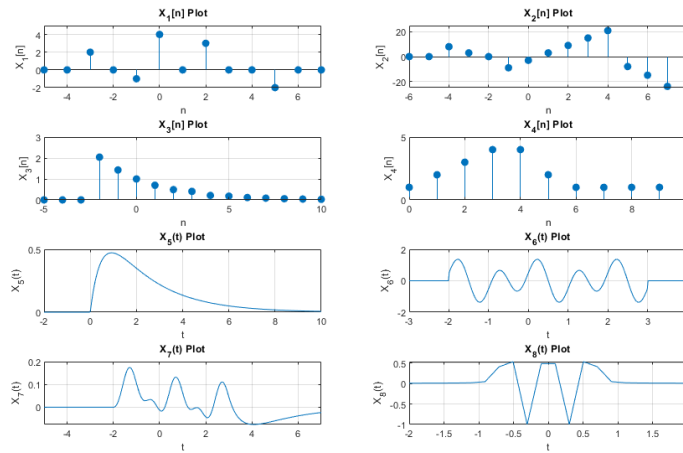


Figure 1: Plots of the discrete and continuous signals.

1.2 Composite Figure and Geometric Shapes

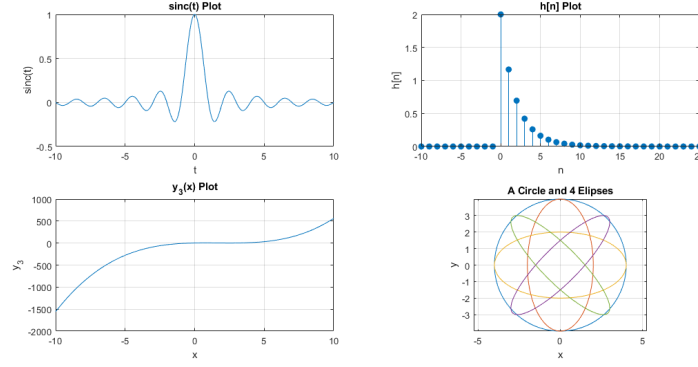


Figure 2: Composite figure showing math functions and geometric shapes.

2 Convolution and Implementation Analysis

2.1 1D Convolution Implementation

To understand the mechanics of linear convolution, the operation $y[n] = x[n] * h[n]$ was implemented in three custom ways without using the built-in `conv` function.

1. Loop-based Implementation (`myConvLoop`): This approach directly implements the convolution sum:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$

Using two nested loops, the outer loop iterates over the output length $L_y = L_x + L_h - 1$, and the inner loop calculates the sum of the products of overlapping elements.

2. Matrix-based Implementation (`myConvMatrix`): Convolution can be modeled as a matrix multiplication $Y = H \cdot X$, where H is a Toeplitz matrix constructed from the impulse response $h[n]$. This method maps the sliding window of convolution into a fixed matrix structure, avoiding nested loops and leveraging MATLAB's highly optimized vector operations.

3. FFT-based Implementation: In the Fourier space convolution turns into multiplication. It is much faster to execute than two previous methods.

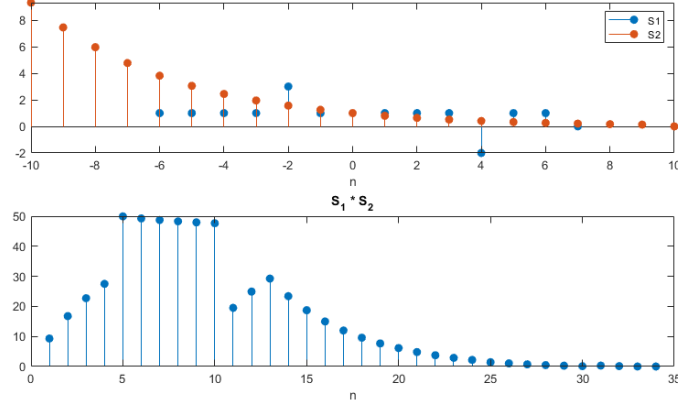


Figure 3: Results of pairwise convolutions using custom functions.

2.2 Timing and Performance Comparison

The execution times of `myConvLoop`, `myConvMatrix` and MATLAB's `conv` were measured using `tic` and `toc` for random vectors of length $N > 500$.

The loop-based method is the slowest because MATLAB is an interpreted language where loop overhead is significant. The matrix method is faster due to internal vectorization. However, MATLAB's built-in `conv` is overwhelmingly the fastest as it is implemented as a pre-compiled, highly optimized C/C++ function, often utilizing Fast Fourier Transforms (FFT) for large arrays to reduce complexity from $O(N^2)$ to $O(N \log N)$.

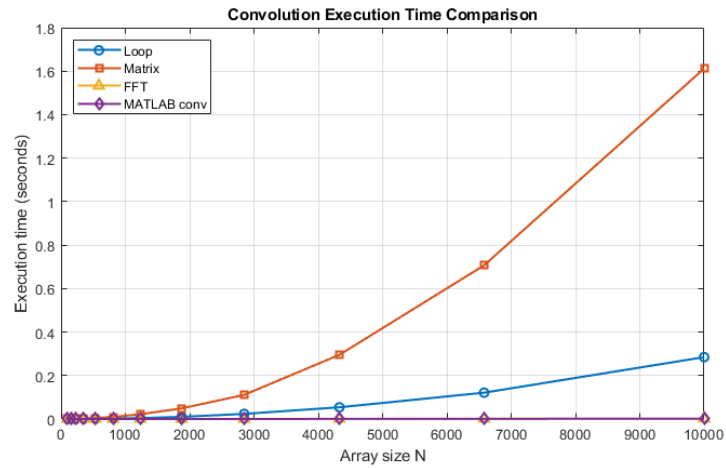


Figure 4: Timing of difference method. As it was expected, the built-in function is much faster than implemented functions.

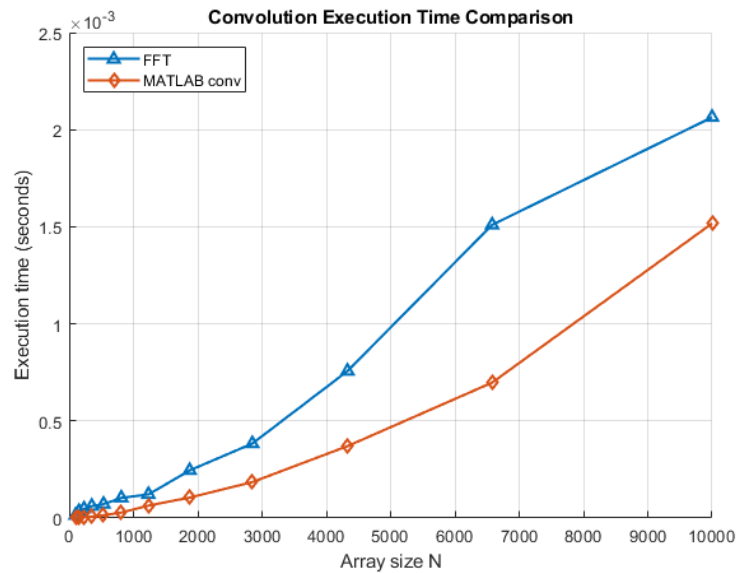


Figure 5: Fourier transform-based function versus built-in `conv` function.

3 Interpolation and Image Processing

3.1 1D Linear Interpolation

To increase the sampling rate of a discrete signal $x[n]$ by a factor of $L = 3$, the signal was first upsampled to create $w[n]$ by inserting $L - 1 = 2$ zeros between every sample of $x[n]$. To perform linear interpolation, this zero-inserted signal $w[n]$ was convolved with an interpolation filter $f[n]$. For linear interpolation with $L = 3$, the ideal discrete filter is a triangular pulse of length $2L - 1$ centered at zero:

$$f[n] = 3 \frac{\sin\left(\frac{\pi}{3} \cdot n\right)}{\pi n}$$

The final interpolated signal is obtained via $y[n] = w[n] * f[n]$. The filter essentially acts as a low-pass filter, smoothing the abrupt changes caused by the zero-insertion.

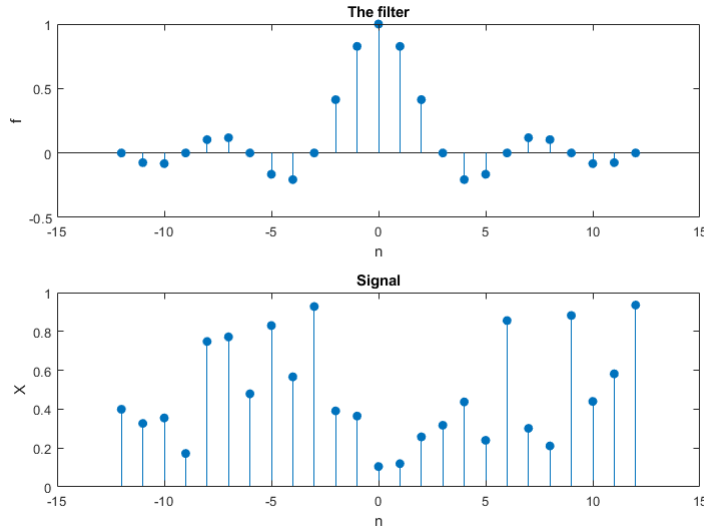


Figure 6: Original signal and filter.

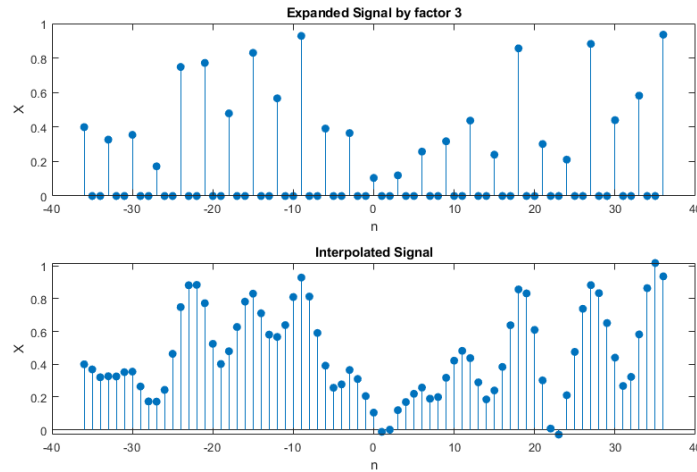


Figure 7: zero-inserted signal, and the linearly interpolated output.

3.2 2D Image Enlargement

The concept of 1D interpolation was extended to 2D image processing. An image was enlarged by a factor of 3 by inserting rows and columns of zeros and convolving the result with a 2D interpolation kernel. The custom convolution approach was compared with MATLAB's built-in `imresize(..., 'bilinear')` function. The results are visually very similar, as both methods mathematically apply bilinear interpolation.



Figure 8: Comparison of custom 2D enlargement and MATLAB's `imresize`.

4 Matched Filter and Edge Detection

4.1 1D Pattern Detection

A matched filter was utilized to find occurrences of a specific pattern $a[n] = [1, -1, -1, 1]$ inside a longer, random binary sequence $x[n]$. The matched filter impulse response $p[n]$ is simply the time-reversed version of the target pattern:

$$p[n] = a[-n]$$

By computing the convolution $y[n] = x[n] * p[n]$, the output is maximized at the exact locations where the pattern aligns with the signal. The peaks in $y[n]$ equal the energy of the pattern, allowing us to successfully report the starting indices of the pattern in $x[n]$.

4.2 Pattern Detection in Images

This concept was applied to 2D arrays to locate a visual pattern inside a noisy binary image. A 2D matched filter (the flipped pattern matrix) was convolved with the image. The output was visualized as a heatmap, where the global maximum indicates the exact center of the detected pattern.

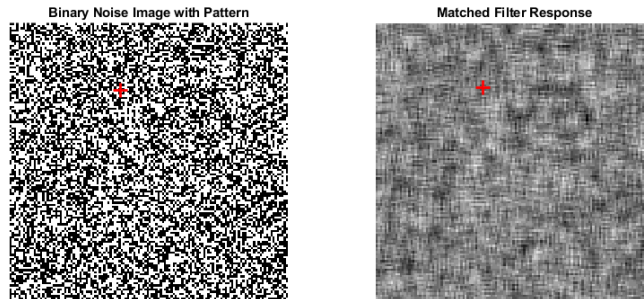


Figure 9: Heatmap of the 2D matched filter output indicating pattern location.

5 Image Processing and Simulation

5.1 Image Processing Basics

A color image was loaded, and its R, G, and B channels were separated and displayed. To convert the image to grayscale, two methods were used: a simple mathematical average of the three channels ($I_{gray} = (R + G + B)/3$), and the built-in `rgb2gray` function. The built-in function yields a perceptually better image because it applies weighted averages (e.g., $0.299R + 0.587G + 0.114B$).

The image was then converted to a binary image using `imbinarize`, which automatically calculates a global threshold (using Otsu's method) to separate pixels into foreground (1) and background (0).



Figure 10: The image decomposed to 3 channel.



Figure 11: Gray scale image with two method.



Figure 12: Binarized image

5.2 Sobel Operator

Edge detection was manually implemented using the Sobel operator. The image was convolved with horizontal (G_x) and vertical (G_y) kernels:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

G_x detects vertical edges by finding horizontal gradients, while G_y detects horizontal edges. The overall gradient magnitude was calculated as $G = \sqrt{G_x^2 + G_y^2}$, which highlights all edges in the image regardless of orientation.

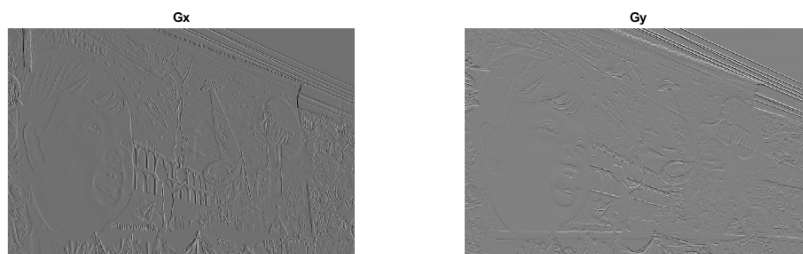


Figure 13: Edge detection showing G_x , G_y .

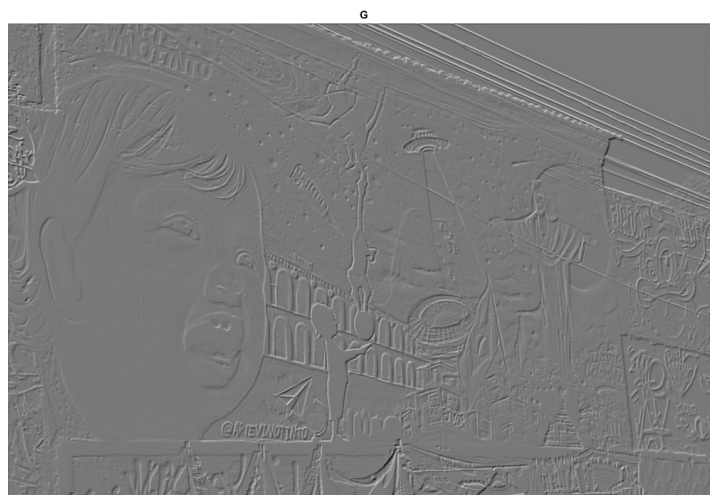


Figure 14: The combined gradient G .

6 Image segmentation

6.1 K-means

An image is essentially a collection of pixels. In a color image, each pixel is a point in 3D space defined by its **R,G,B** color values. We want to group these pixels into *clusters* such that pixels in the same cluster have similar colors. Every pixel in the image is assigned to the nearest centroid based on the Euclidean distance between their color values. Once all pixels are assigned, the new center of each cluster is recalculated by taking the average color of all pixels currently assigned to that cluster.



Figure 15: Image segmentation with K-means method($K = 3$).

6.2 Otsu's method

The algorithm assumes that the image contains two classes of pixels: foreground and background. It aims to find a threshold level **k** that partitions the grayscale histogram into these two classes such that the variance between the two classes is maximized.



Figure 16: Image segmented into foreground and background.

6.3 Comparison: K-means and Otsu

1. **Sensitivity to initial values:** K-means depends on the initial cluster centers and may converge to different results on different runs. Otsu doesn't need any initial values.
2. **Number of Segments:** K-means, can determine K segments, where it is chosen by user. Otsu method is for binary segmentation, foreground and background.
3. **Compatible with color images:** K-means works with both grayscale and color images. Otsu method is most commonly used for grayscale images.
4. **Timing and Performance:** Otsu is usually faster and simpler because it only computes one optimal threshold from the histogram. Very efficient for binary segmentation.

7 Voting Simulation

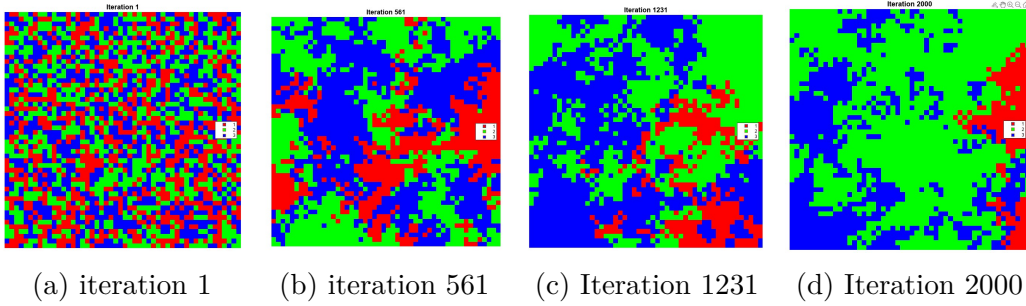


Figure 17: Voting Simulation. In different iterations. It seems like it goes from complete chaos to a more stable point as iterations pass.

8 Functions in Detail

8.1 K-Means Clustering Implementation

The function `segmentImage` performs image segmentation by clustering pixels in a multidimensional color space. The implementation follows the standard K-means algorithm approach:

1. **Data Reshaping:** We transform the 2D/3D image into a 2D matrix of size $(N \times C)$, where $N = \text{rows} \times \text{cols}$ and C is the number of channels (e.g., 3 for RGB). This allows us to treat each pixel as a single point in an C -dimensional space.

```
1 pixelList = reshape(img, rows * cols, channels);
```

2. **Distance Calculation:** In each iteration, we calculate the Euclidean distance between every pixel and the k current cluster centers using `pdist2`. This generates a distance matrix of size $(N \times k)$.

$$d(p, c) = \sqrt{\sum_{j=1}^C (p_j - c_j)^2} \quad (1)$$

3. **Assignment and Update:**

- **Assignment:** Each pixel is assigned to the cluster with the minimum distance: `labels = min(distances, [], 2)`.
 - **Update:** The centroids are recalculated as the mean of all pixels assigned to that cluster. The check `if sum(labels == i) > 0` ensures we do not attempt to calculate the mean of an empty cluster.
4. **Convergence Criteria:** We monitor the displacement of the centers between iterations using the Frobenius norm. If the centers shift by less than 0.1, the algorithm has converged, and we exit the loop early:


```

1  if norm(centers - oldCenters) < 0.1
2      break;
3  end

```
 5. **Reconstruction:** Finally, we replace each pixel's original color with its corresponding cluster center and reshape the list back into the original image dimensions.

8.2 Otsu's Thresholding Implementation

The function `otsu_segmentation` implements the Otsu method to determine an optimal global threshold by maximizing the between-class variance. The implementation follows these logical steps:

1. **Data Preparation:** The input is cast to `double` to ensure precision during division operations. We calculate the normalized histogram (probability distribution) p_i of the grayscale intensities:


```

1  [counts, ~] = imhist(uint8(grayImage));
2  probs = counts / numPixels;

```
2. **Cumulative Calculations:** We compute the cumulative sums of the probabilities (ω) and the intensity values (μ) to allow for fast computation of class statistics for every possible threshold level t :


```

1  cumSum = cumsum(probs);
2  cumMean = cumsum((0:255)' .* probs);

```

3. **Class Statistics:** For a threshold t , the two classes (background and foreground) have probabilities $\omega_1(t)$ and $\omega_2(t)$, and means $\mu_1(t)$ and $\mu_2(t)$. We calculate these using:

$$\mu_1(t) = \frac{\sum_{i=0}^t i \cdot p_i}{\omega_1(t)}, \quad \mu_2(t) = \frac{\mu_{total} - \sum_{i=0}^t i \cdot p_i}{1 - \omega_1(t)} \quad (2)$$

The term `eps` is added to the denominators to prevent division by zero in regions where the histogram may be empty.

4. **Optimization:** We maximize the between-class variance:

$$\sigma_b^2(t) = \omega_1(t)\omega_2(t)[\mu_1(t) - \mu_2(t)]^2 \quad (3)$$

The index corresponding to the maximum variance is selected as the optimal threshold. Since MATLAB indices are 1-based (starting at 1) and intensity values are 0-based (starting at 0), we adjust the result:

```
1 [~, thresholdIndex] = max(betweenClassVariance);
2 threshold = (thresholdIndex - 1) / 255;
```

5. **Binarization:** Finally, the image is segmented using the calculated normalized threshold.

8.3 Voting Simulation System

1. **Neighbor Detection (`get_neighbors`):** This utility function defines the connectivity of the grid. It uses a 2D stencil approach to identify valid 8-connected neighbors for any given cell (i, j) .

```
1 directions = [-1, -1; -1, 0; -1, 1; 0, -1; 0, 1; 1, -1; 1, 0; 1, 1];
```

Boundary conditions are enforced by checking if the calculated neighbor indices (ni, nj) fall within the range $[1, n]$.

2. **Stochastic Dynamics (`voting_simulation`):** The core simulation logic mimics a *voter model*. In each iteration:

- We select a subset of cells based on the parameter k , representing the intensity of interaction per iteration.

- A randomly selected neighbor of the active cell adopts the active cell's *vote* (state). This causes clusters of identical values to grow and compete, eventually leading to *opinion islands*.

3. **Visualization (`display_votes`):** To render the simulation, the integer-based `votes` grid is mapped into an RGB image. Each integer state (1, 2, or 3) corresponds to a specific row in a color definition matrix:

```
1 img(i, j, :) = colors(votes(i, j), :);
```